

Documentacion Tecnica

Actividad 3

Alumno: Kevin Alejandro Hernandez Quintana

1. Introducción

La actividad resuelve tres desafíos en Java mediante recursividad, divide y vencerás y backtracking, los dos primeros desafíos producen una contraseña, el tercero utiliza esa idea como punto de partida para buscar la ruta más corta en un laberinto, la solución funciona con diferentes entradas, evita ciclos, compara rutas y detecta cuando no existe una salida.

2. Organización del programa

El código se dividió en clases para separar las responsabilidades, desafio1 realiza la reducción recursiva de números, desafio2 obtiene el valor máximo mediante divide y vencerás, SolucionadorLaberinto ejecuta el backtracking, posicion representa una coordenada y ResultadoLaberinto almacena la ruta, las posiciones visitadas y las métricas, Main ejecuta los tres desafíos y Pruebas comprueba los casos requeridos.

Desafio 1

El primer desafío recibe un entero, suma sus dígitos de forma recursiva y repite el proceso hasta obtener un solo dígito, el método sumarDigitos separa el último dígito mediante el residuo de la división entre diez y llama nuevamente al método con el resto del número.

Caso base

El caso base de reducirAUnDigito ocurre cuando el valor se encuentra entre menos nueve y nueve, porque ya contiene un solo dígito, en ese momento se devuelve su valor absoluto, el caso base de sumarDigitos ocurre bajo la misma condición y devuelve el dígito restante.

Caso recursivo

Si el número contiene más de un dígito, sumarDigitos obtiene el último dígito con el operador de residuo y continúa con la división entera entre diez

reducirAUnDigito recibe la suma y se llama otra vez si el resultado todavía tiene más de un dígito.

Resultados del desafío 1

84729: $8 + 4 + 7 + 2 + 9 = 30$; $3 + 0 = 3$.

56318: $5 + 6 + 3 + 1 + 8 = 23$; $2 + 3 = 5$.

92746: $9 + 2 + 7 + 4 + 6 = 28$; $2 + 8 = 10$; $1 + 0 = 1$.

Clave 1: 351.

Desafio 2

El segundo desafío busca el valor máximo del arreglo de lecturas sin ordenarlo y sin utilizar un ciclo tradicional para buscar el mayor, la solución aplica divide y vencerás sobre un intervalo definido por sus índices de inicio y fin.

División

El método calcula el punto medio del intervalo, la parte izquierda comprende desde el inicio hasta el punto medio y la parte derecha comienza en la siguiente posición y termina en el índice final, cada mitad se divide de la misma manera hasta llegar a intervalos de un elemento.

Resolución y combinación

Un intervalo de un elemento constituye el caso base, porque ese elemento es el máximo de su propio subproblema, al regresar de las llamadas recursivas se comparan los máximos de las dos mitades, el mayor se devuelve al nivel anterior hasta obtener el máximo del arreglo completo.

La lectura máxima del arreglo es 947, por lo tanto, la clave 2 es 947 y la contraseña formada con ambas claves es 351-947.

Desafio 3

El laberinto se representa con una matriz de caracteres, S identifica el inicio, E representa la salida, el punto indica una posición disponible y el signo de número representa un obstáculo, antes de iniciar la búsqueda, el programa verifica que el

laberinto no esté vacío, que sea rectangular, que contenga símbolos válidos y que tenga exactamente un inicio y una salida

Funcionamiento del backtracking

La exploración comienza en S y prueba los movimientos hacia la derecha, abajo, izquierda y arriba, al entrar a una posición, el algoritmo la registra, la agrega a la ruta actual y la marca como parte del camino, después explora recursivamente cada movimiento válido, cuando termina una alternativa, elimina la posición y deshace la marca, este retroceso permite regresar desde un callejón sin salida y probar otro camino.

Control de ciclos

La matriz enRutaActual indica qué posiciones forman parte del camino que se está explorando, antes de avanzar, el programa comprueba que la coordenada esté dentro del laberinto, que no sea un obstáculo y que no aparezca en la ruta actual, la marca se elimina al retroceder, de esta forma, una celda puede probarse desde otra alternativa, pero no puede repetirse dentro del mismo camino.

Selección de la ruta más corta

La búsqueda no termina al encontrar la primera salida, cada vez que llega a E, el algoritmo guarda una copia de la ruta si no existe una solución anterior o si la nueva ruta contiene menos posiciones, el número de movimientos corresponde a la cantidad de posiciones menos uno, porque la posición inicial no cuenta como desplazamiento.

Poda de la búsqueda

La optimización compara la longitud de la ruta actual con la mejor solución conocida, si la ruta ya iguala o supera esa longitud, deja de explorarla, todos los movimientos tienen el mismo costo, por lo que una ruta que ya es igual o más larga no puede convertirse en una solución mejor, la poda evita trabajo innecesario sin afectar la mejor ruta.

Caso sin solución

Si todas las alternativas terminan sin alcanzar E, la mejor ruta permanece vacía, el programa informa esta situación mediante el texto RUTA:SIN_SOLUCION y asigna menos uno al número de movimientos.

Pruebas realizadas

La clase Pruebas ejecuta 16 verificaciones, incluye los bloques numéricos, casos especiales para la reducción, arreglos con un elemento o números negativos y los cinco escenarios de laberinto solicitados.

Caso 1 Ruta directa

S..E

El resultado es una ruta directa de tres movimientos.

Caso 2 Callejón sin salida

S..#

..#.#

...E

La rama superior queda bloqueada, el algoritmo retrocede y encuentra una ruta de cinco movimientos.

Caso 3 Varias rutas

S..

..#.

..E

Existen caminos alrededor del obstáculo central, la longitud mínima obtenida es de cuatro movimientos.

Caso 4 La primera ruta no es la más corta

S....

.###.

E....

La primera alternativa que comienza hacia la derecha, utiliza diez movimientos, la búsqueda continúa y encuentra una ruta de dos movimientos hacia abajo, dos es el valor mínimo posible porque el inicio y la salida están separados por dos filas en la misma columna, este caso demuestra que el programa compara soluciones y no conserva automáticamente la primera

Caso 5 Sin solución

S#E

.##

...

La salida está aislada por obstáculos, el programa explora los caminos disponibles y determina que no existe una ruta válida.

Las 16 verificaciones terminaron correctamente, las capturas de la ejecución principal y de las pruebas completas se encuentran en la carpeta de evidencias del proyecto.

Decisiones de implementación

Cada desafío se colocó en una clase independiente para separar su lógica, las coordenadas se representan con Posicion y el resultado completo se almacena en ResultadoLaberinto, las listas del resultado se copian para evitar modificaciones posteriores, la marca de posiciones pertenece a la ruta actual y se elimina durante el retroceso, lo que permite comparar caminos distintos sin crear ciclos.

Reflexión final

Diferencia entre recursividad e iteración

Una solución recursiva llama al mismo método con entradas cada vez más pequeñas y necesita un caso base para terminar, una solución iterativa repite instrucciones mediante ciclos y controla su estado con variables, ambas pueden obtener el mismo resultado, aunque representan el proceso de forma diferente.

Utilidad de divide y vencerás

Divide y vencerás facilita un problema cuando puede separarse en subproblemas del mismo tipo, cada parte se resuelve por separado y los resultados se combinan, en esta actividad, comparar los máximos de dos mitades permite obtener el máximo del arreglo completo.

Una solución no siempre es la mejor

La primera ruta encontrada depende del orden de los movimientos, puede llegar a la salida y aun así utilizar más pasos que otra alternativa, por esta razón, el backtracking debe continuar y comparar las rutas antes de decidir cuál es la mejor

Consecuencia de no controlar posiciones visitadas

Sin el control de posiciones, el algoritmo podría regresar continuamente a celdas anteriores y formar ciclos, esto generaría llamadas recursivas repetidas, aumentaría el uso de memoria y podría provocar un desbordamiento de la pila.

Aprendizaje sobre la ruta más corta

Buscar la ruta más corta requiere conservar resultados, comparar longitudes y continuar después de encontrar una salida, también exige justificar las optimizaciones para no eliminar una alternativa que podría mejorar la solución actual.

Resultados

La clave 1 es 351, la clave 2 es 947 y la contraseña es 351-947 , las 16 pruebas fueron superadas.